Advanced Tools for Complex Network Analysis CNET 5052, Spring 2026, Assignment 4

Colin White

1 Network reconstruction

Describe your choice of graph and dynamics, Visualize one of the time series matrices: I chose an erdos renyi graph with the parameters 64 nodes, probability for edge creation 0.1, and seed=42. I applied Kuramoto as the dynamic function from netrd. Kuramoto oscillators model synchronization processes. Each oscillator has its own natural frequency, but interacts with others through a coupling term that nudges them toward a common phase. Wikipedia states the following The Kuramoto model makes several assumptions, including that there is weak coupling, that the oscillators are identical or nearly identical, and that interactions depend sinusoidally on the phase difference between each pair of objects.

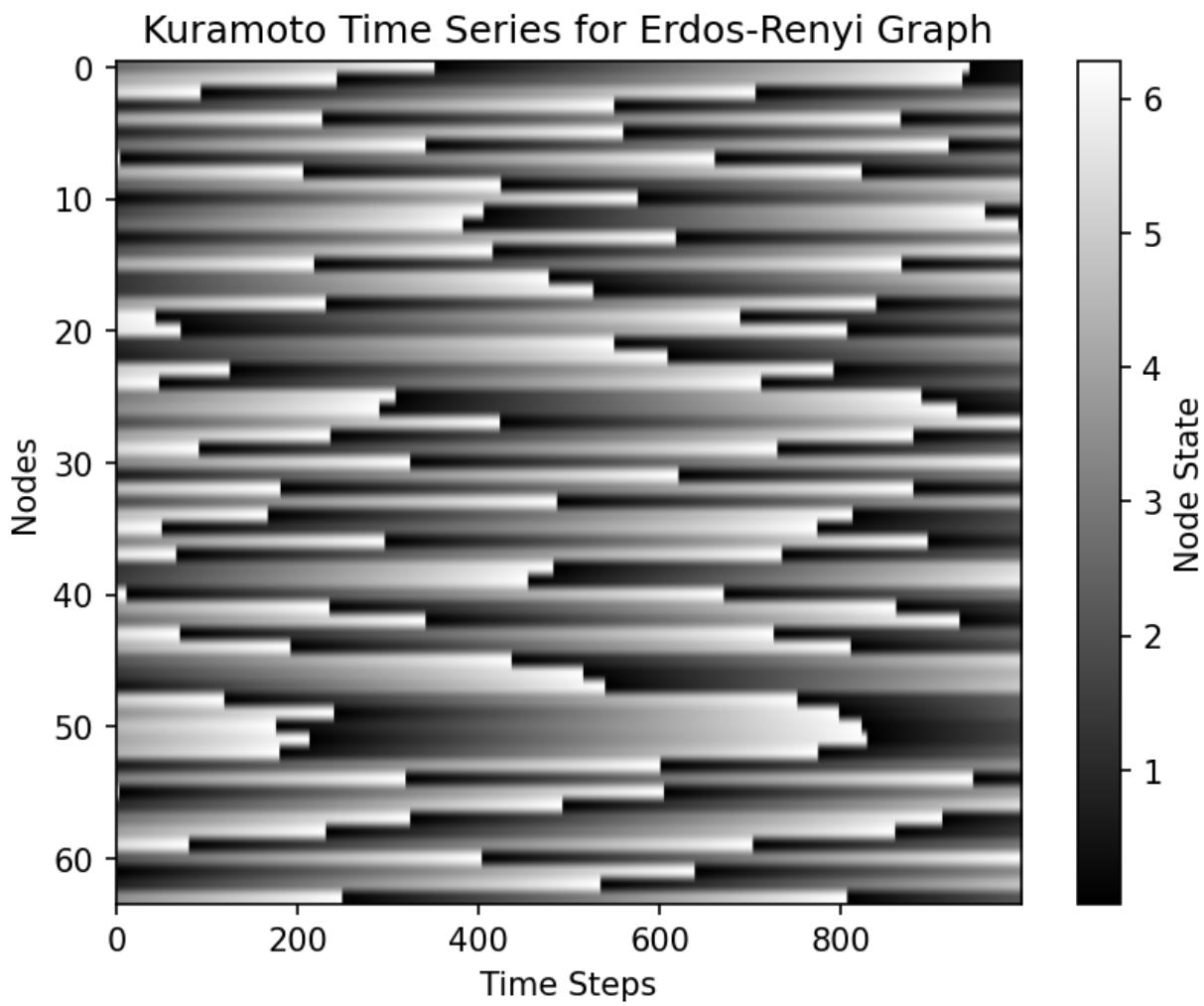

Figure 1: One ER graph with a Kuramoto dynamic applied for 1000 time steps

Introduce a measure, D(G, R), that takes two graphs as input—your original network G and a reconstructed (and thresholded) network R—and outputs a single scalar value. Use this function to quantify how "close" each of your 20 reconstructed networks are to the original graph. Create a bar plot of these values with 20 bars, corresponding to the D(G, Ri) of each of your reconstructed networks, Ri.

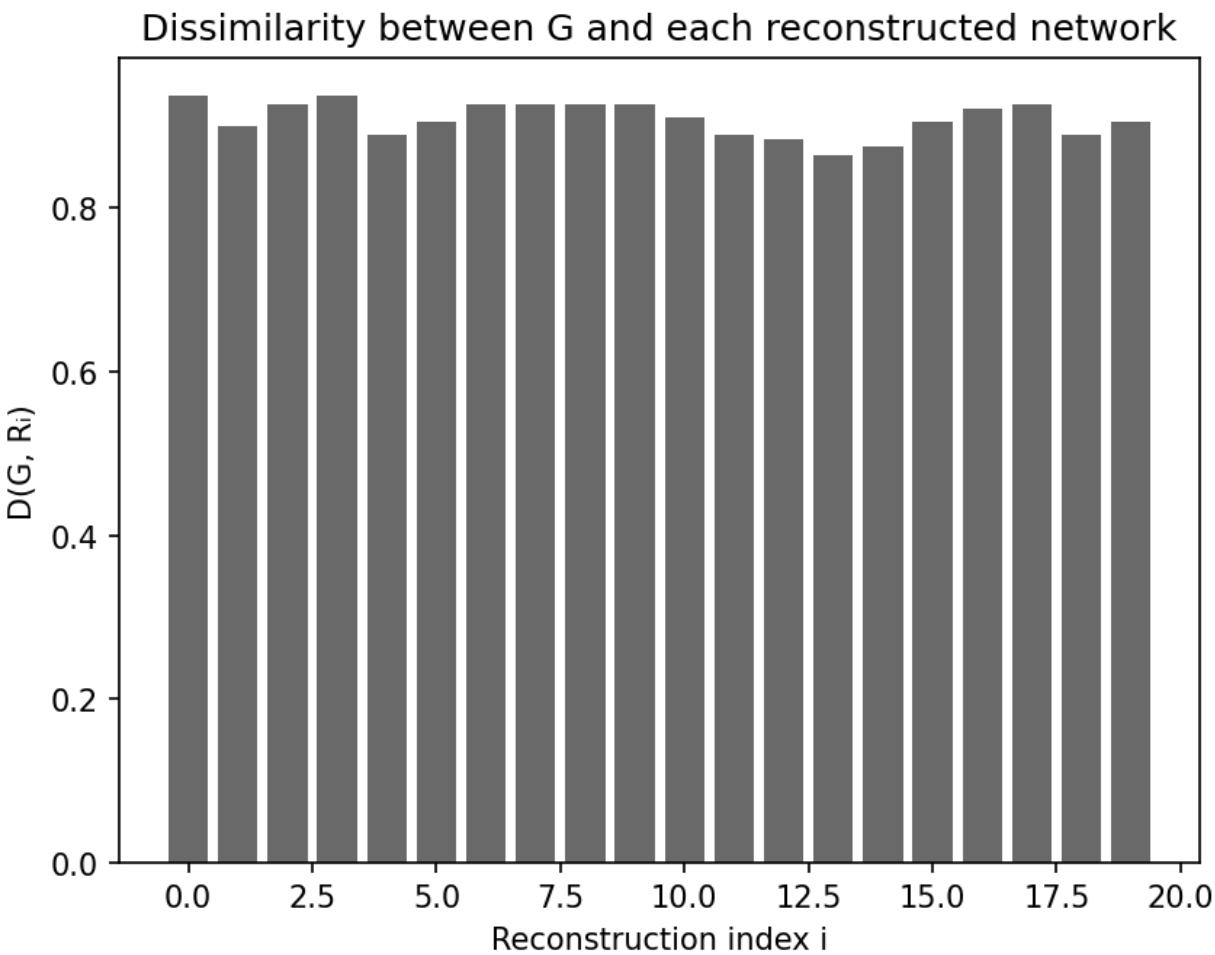

Figure 2: D(G, Ri) of each reconstructed networks, Ri. D(G, R), takes two graphs as input, the original network G and a reconstructed (and thresholded) network R—and out-puts a single scalar value

Use your dissimilarity measure D to compute D(Ri, Rj) between each pair of the 20 reconstructed networks you have created. Put these into a distance matrix d, such that dij corresponds to how different reconstructed networks Ri and Rj are from one another

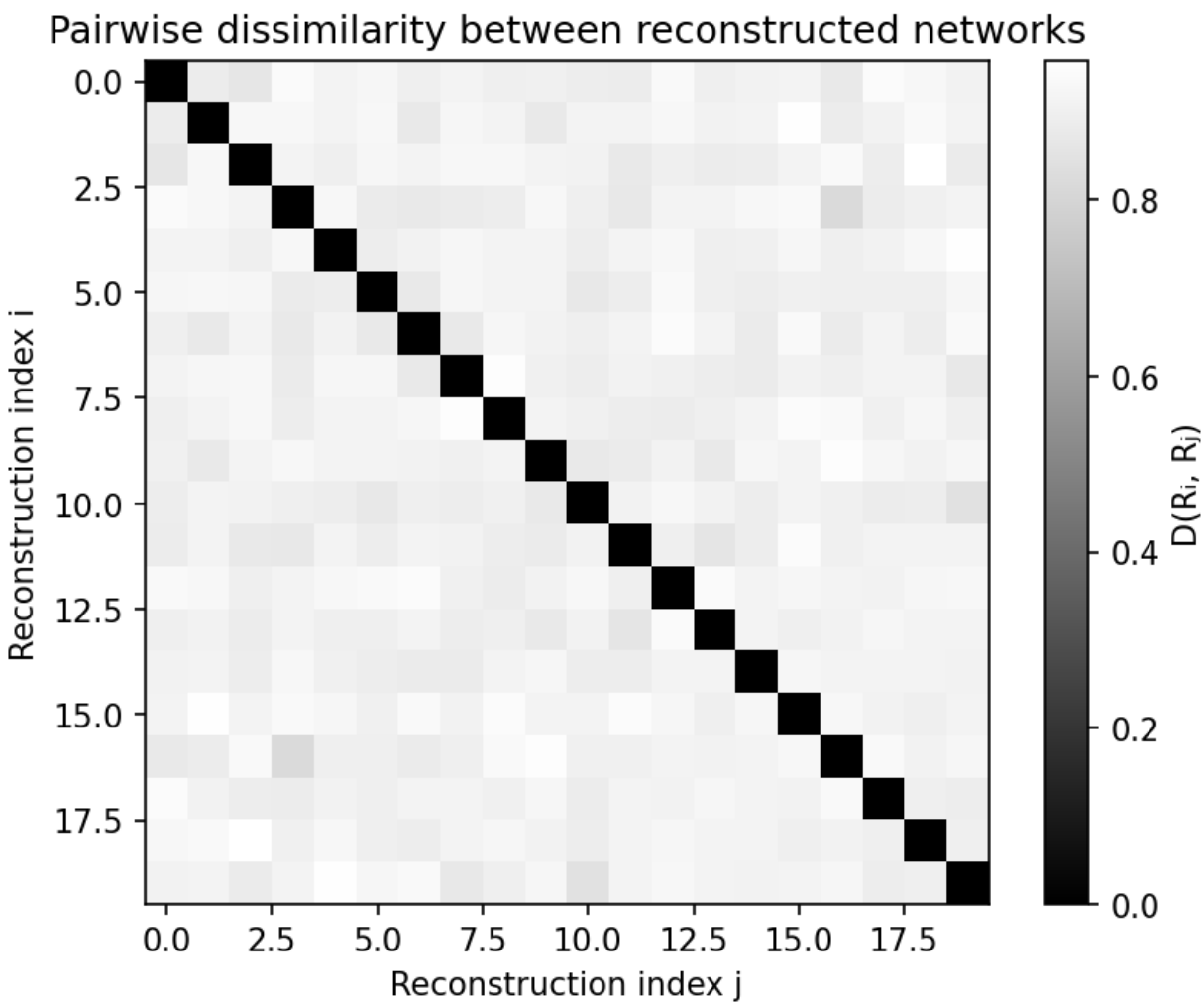

Figure 3: Distance betwen D(Ri, Rj) compared between each pair of the 20 reconstructed networks

What would it mean if all of your reconstructed networks are all very close or similar to one another? What if they were far/dissimilar? How might the choice of dynamical process, original network G, reconstruction algorithm, and/or thresholding approach all influence how effective or ineffective a given reconstruction technique is?

What would it mean if all of your reconstructed networks are all very close or similar to one another?

If all 20 reconstructed networks were very similar to one another it would indicate that the reconstruction algorithm is finding a consistent signal in spite of all the noise in the data. Each time series, despite being generated with different random seeds, would be pointing the algorithm toward the same suspected graph structure. This is a sign that the dynamics applied do not obscure the underlying network beyond reconstruction by the chosen method of reconstruction.

What if they were far/dissimilar?

If the reconstructions are far from one another like in my data here, with most pairwise distances

near 0.85 means my application of the algorithm is very sensitive to noise. Different seeds produce very different suspected graphs, none of which are very close to G or to each other. So the way I applied the reconstruction method may not be well matched to see the graph past the obscuring dynamics, it is seeing mostly noise, not recovering genuine signal.

How might the choice of original network G influence how effective or ineffective a given reconstruction technique is?

The ER graph used has a pretty homogeneous degree distribution, which may be harder to reconstruct because the dynamics are evenly distributed. A BA graph with hubs could be easier, since hub nodes could be like anchor points. It would depend on which dynamic process was happening to the graph.

Reconstruction algorithm?

Reconstruction algorithms are tools. Like any tool, each has its purpose, and context dependent strengths and weaknesses. There's no universally best tool or algorithm. The underlying graph structure, noise characteristics, type of dynamics, and size of the time series all inform what reconstruction algorithm would be best. Based on research a method like partial correlation, would probably have done better at reconstructing my original ER graph but my version of netrd had compatibility issues with that method.

Thresholding?

Thresholding by average degree forces the reconstructed network to have the same overall density as the original graph. An alternative would be to threshold by statistical significance.

2 Big data and Bloom Filters

For all possible edges, how long would it take to check for their existence in the complete graph? How long would it take to check if those edges exist if you first reference your bloom filter? How does this answer change as you adjust apsilon from 0.01 to 0.05 to 0.1?

Without a filter checking all aprox 500,000 possible edges takes 138.8 hours at 1 second per lookup. With a bloom filter of various epsilons here is how long it takes. With epsilon at 0.01, it takes 11.6 hours and creates few false positives. With epsilon at 0.05 it take 17.7 hours but with more false positives. With epsilon at 0.1 it would take 23.9 hours with the most false positives.

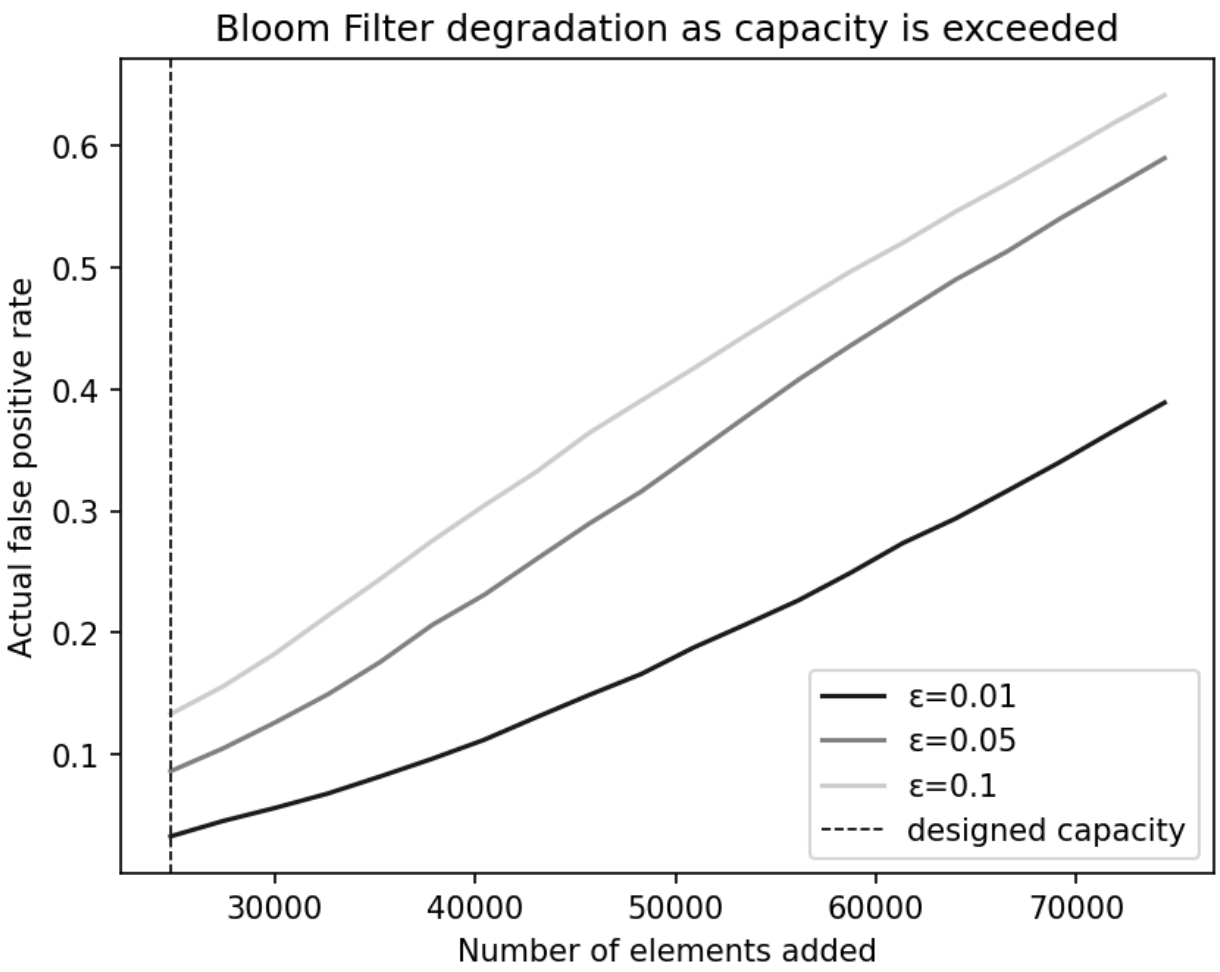

Figure 4

When a Bloom Filter is filled beyond its designed capacity n, the bit array m becomes increasingly saturated, Meaning more bits are set to 1, so random queries are more likely to find all k of their hash positions already set, producing false positives. The false positive rate climbs well above the target epsilon, and the filter loses its value as a pre-cache. Filters designed for lower epsilon degrade slower but all three converge toward high false positive rates by 3x overfill. This means bloom filters require a reasonably good estimate of n upfront because overestimating is ok, but underestimating by say half degrades performance quite a bit.

3 ABMs: exploration, exploitation, and network structure

i How do they operationalize exploration and exploitation? Here they define exploitation as when an agent copies a better-performing neighbor. Exploration is when no better neighbor exists and the agent instead tries a random one bit mutation of their current solution, but only if it improves fitness. It's trying something not tested before. Every agent does one or the other each time step.

ii What is an NK landscape? It's the fitness landscape aka a rugged landscape with peaks and valleys. Each point is identified with a binary strings of length n. Each string represents a candidate solution. K controls how rugged the landscape is. When K is at 0 there's one smooth global peak and hill-climbing always works. At high K the landscape is jagged with many local optima and agents easily get stuck.

iii Describe Figure 4. The y-axis is average fitness score normalized to the actual optimum, across 1000 simulation runs. The x-axis is time steps, there are 100 time steps. Two lines are compared: a linear or path graph, and a fully connected graph. The complete graph rises faster earlier on but plateaus at a lower level. The linear graph rises more slowly but eventually surpasses the complete graph and reaches a higher fitness.

iv Describe Figure 5. This plots the average number of unique solutions held by agents over time for both networks. The complete graph converges on 1 or 2 unique solutions very quickly. The path graph converges much slowler, and keeps many more unique solutions for much longer, this is because it explores more because it has fewer neighbors.

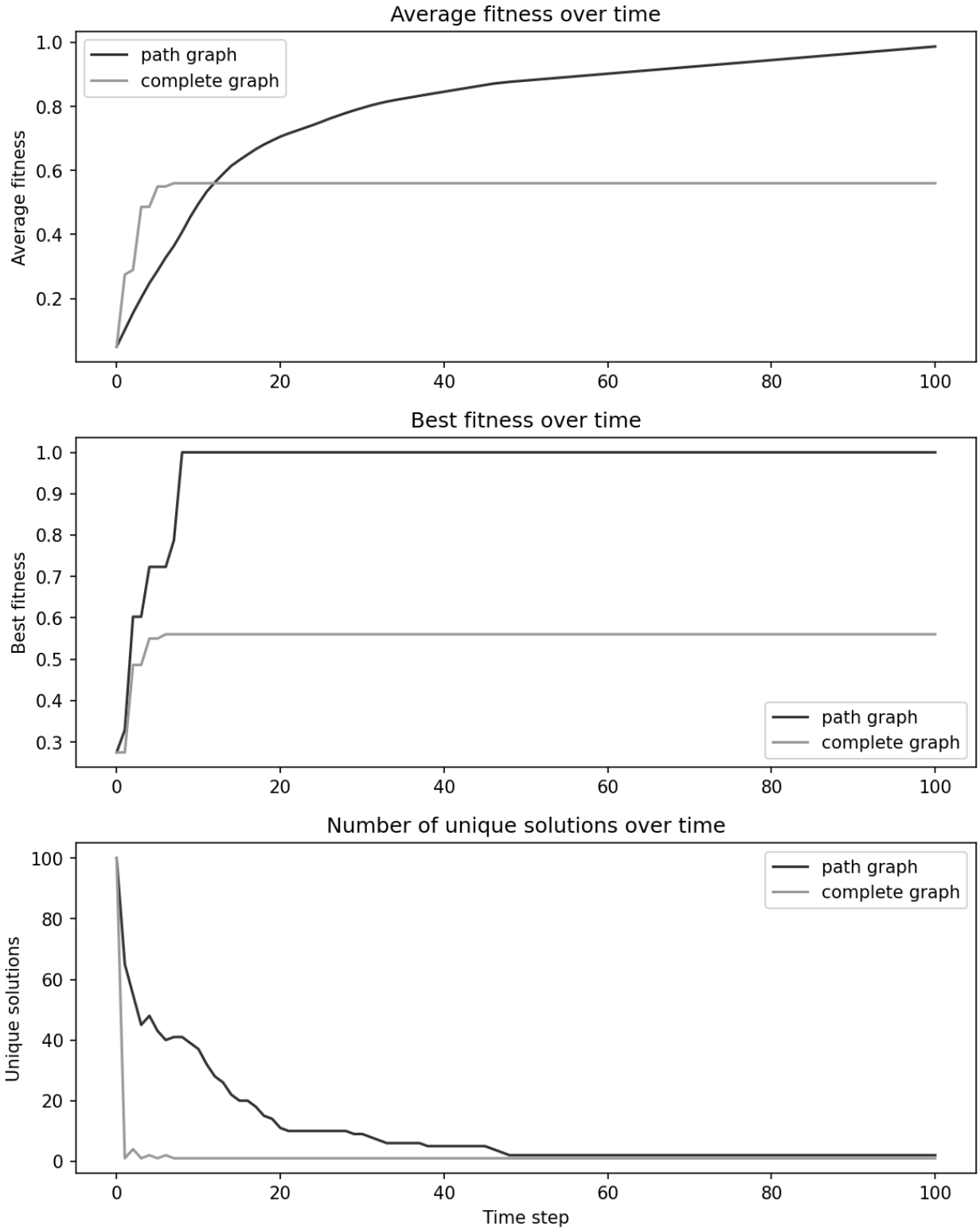

Figure 5: Single run comparison, Agent based model on two networks with the same number of agents: a path graph and a complete graph. Use 100 agents, N = 20, and K = 5. Start each agent at a random binary string.

The complete graph improves more quickly in the short run. It rapidly converges to a single solution, getting stuck at a local optimum around fitness 0.55. The path graph is slower initially but keeps diversity much longer, allowing continued exploration of the landscape. By about t=10 the path graph surpasses the complete graph for average fitness and by t=100 reaches perfect fitness in best and average category. The diversity plot hints why. The complete graph converges on 1 unique solution too soon while the path graph takes time to explore more.

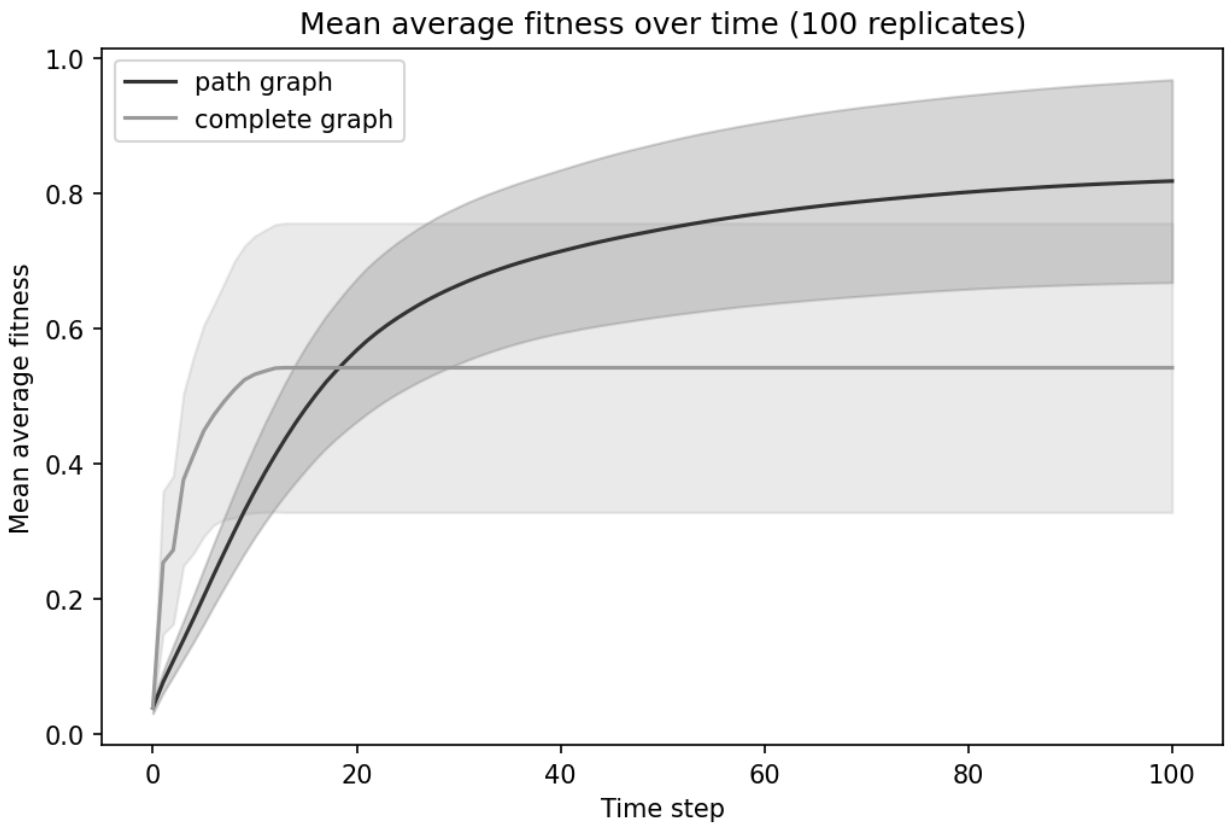

Figure 6: replicating Figure 4 from Lazer and Friedman (2007

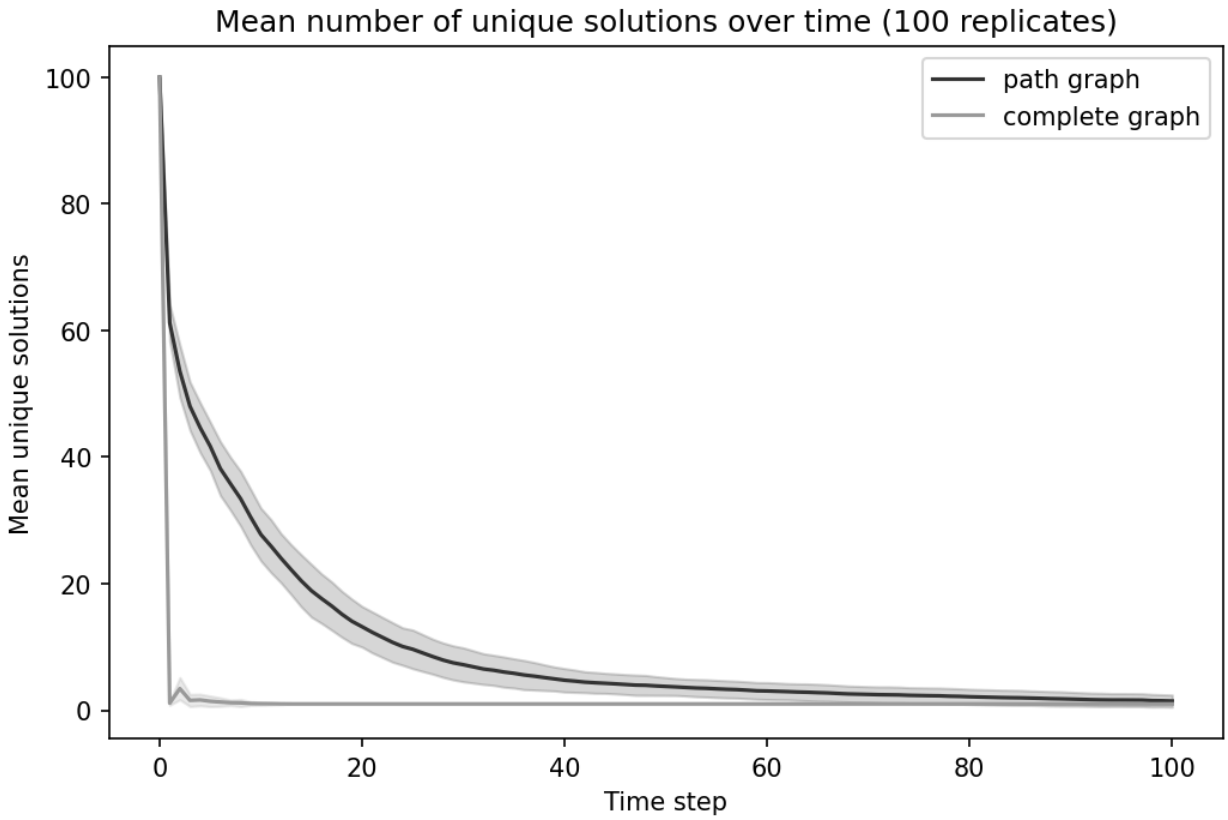

Figure 7

The complete graph rises faster in the short run, while the path graph lags behind. The complete graph plateaus after t=15 and stops improving at a lower overall fitness. The path graph continues to improve the entire simulation. The complete graph converged to a small number of unique solutions almost instantly because it eliminates diversity so quickly. The path graph converges much more slowly. It only fully converges around near time 80. This extra exploration time is why it finds a better solution.

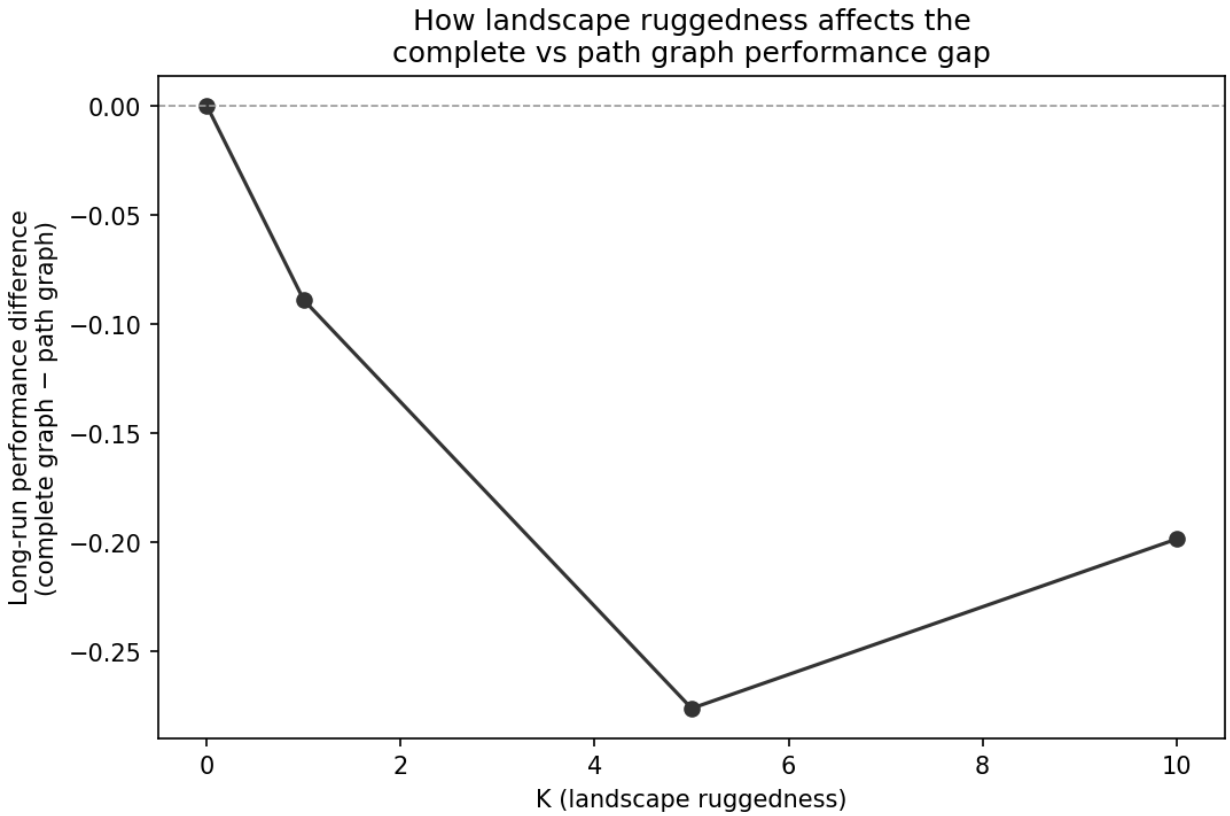

Figure 8: Ruggedness comparison

The y-axis shows complete graph minus path graph long-run performance. Values below zero mean the path graph wins. The dashed line at zero is where the two networks perform equally.

When K is 0 the difference between the graphs is exactly 0. Both networks find the global optimum every time. This is because the landscape has no ruggedness and a single peak. Simple hill climbing works every time regardless of network structure. When K is 1 the Path graph is already noticeably better than the complete graph. This shows even minimal ruggedness is enough to trap the complete graph. The complete graph converges too quickly onto whichever local optimum its best initial agent started at. When K is 5 the path graph has the largest advantage. The path graph's slow convergence lets it explore many peaks before settling. When K is 10 The path graph has less of a relative advantage. I think this is because the landscape is so rugged that even the path graph stays confused. The hills are too chaotic, and unpredictable to climb.

Running many replicates is essential because a single simulation is highly sensitive to the specific NK landscape and the specific initial population. Only by averaging over many independent replicates can you separate the effect of network structure from the noise introduced by random landscape generation and random initial conditions. Holding the landscape and initial population fixed within each replicate when comparing networks is equally critical. If the path graph and complete graph saw different landscapes or started from different positions, any observed performance difference could be due to one network getting a luckier landscape rather than the network

structure itself. By holding these fixed and only varying the graph, the comparison becomes a true controlled experiment.

Measuring solution diversity in addition to fitness matters because fitness alone can be misleading. A population with high average fitness but only one unique solution has converged and cannot improve further — it is stuck. A population with lower average fitness but many unique solutions is still exploring and may reach higher fitness later. It helps you know if your model got stuck in a local optimum, and benchmark if it is done exploring yet.

Further research questions could be: What other graph types could we try? Could we use multiple different graphs exploring the same space? If so, should they overlap in area of exploration not at all, partially or totally? What if we gave them a third option, exploit, explore, or follow some 3rd rule. Could we add more randomness to their exploration mutation step?

Thank you.